# Parallel algorithm design

# Parallel execution in modern processors

- Multicores
  - Multiple execution cores
  - Thread level parallelism
- SIMD
  - Multiple ALUs that execute the same instruction stream
  - Efficient design for data-parallel workloads
- Superscalar
  - Exploit ILP within an instruction stream
  - Parallelism automatically and dynamically discovered by the hardware during execution
- Hardware-supported multi-threading
  - Context of threads managed in HW
  - Execute instructions from several threads at the same time

# Parallel programming models

- Describe concurrent, parallel computations through:
  - Threads
  - Specific language constructions
- Programming models
  - **Shared address space:** threads use shared variables
  - **Message passing:** threads/processes communicate through messages
  - **Data parallel:** structure computation as a map over a collection of data, limit communication

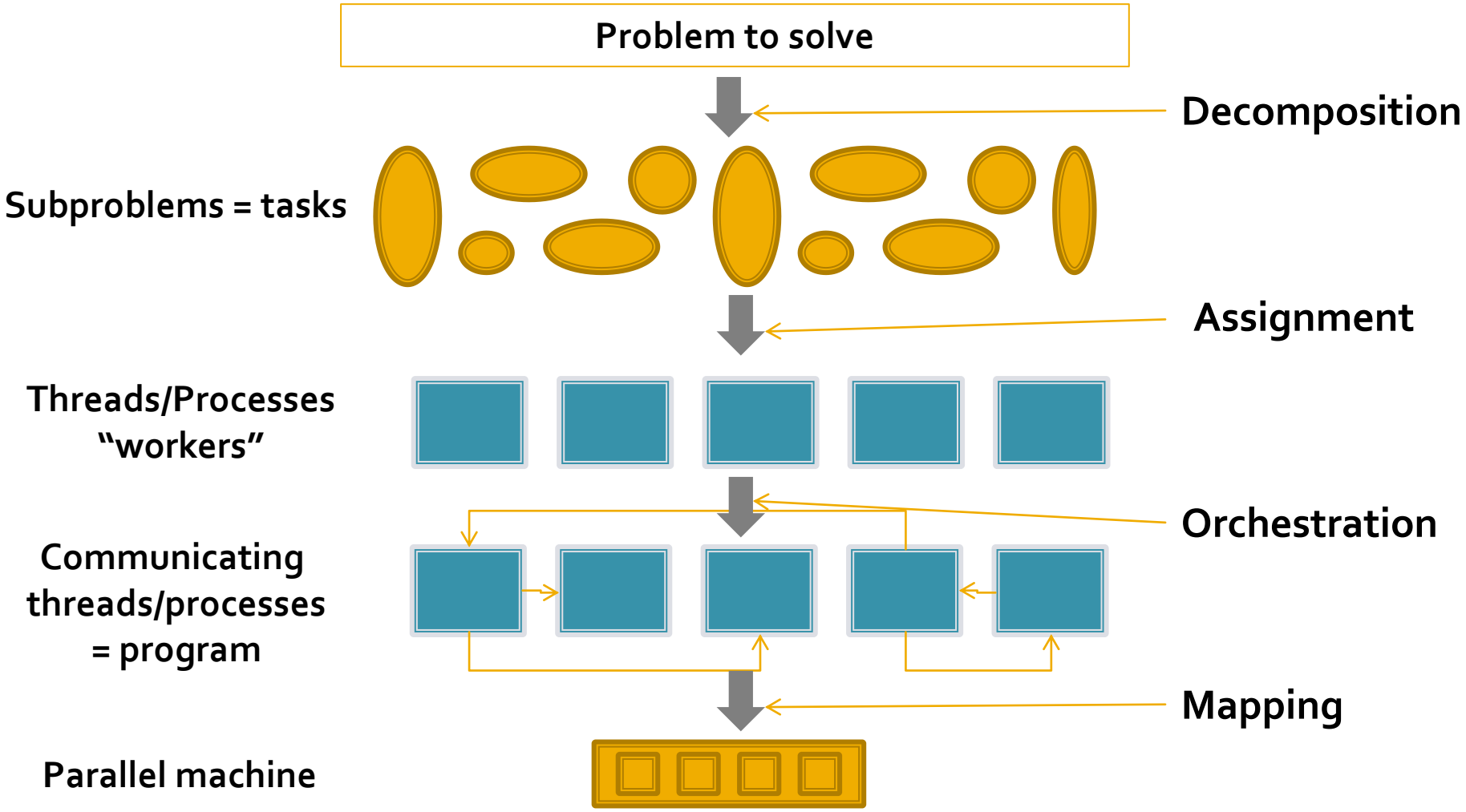
# Goals for designing parallel programs

- Decomposing work into pieces that can be performed in parallel
- Assigning work to processors
- Managing communication/synchronization between the processors

- Achieve speedup:

$$\text{speedup} = \frac{\textbf{Execution time (1 processor)}}{\textbf{Execution time (P processors)}}$$

# Creating a parallel program



# Concepts

- Task = piece of work done by the program
  - Smallest unit of concurrency
  - Granularity (amount of work): fine-grained; coarse-grained
- Thread/Process = abstract entity that performs tasks ("worker")
  - Are assigned one or more tasks
  - Communicate and synchronize with one another
- Processor = executes threads/processes
  - Number of processes ≠ number of processors

# Parallelization process

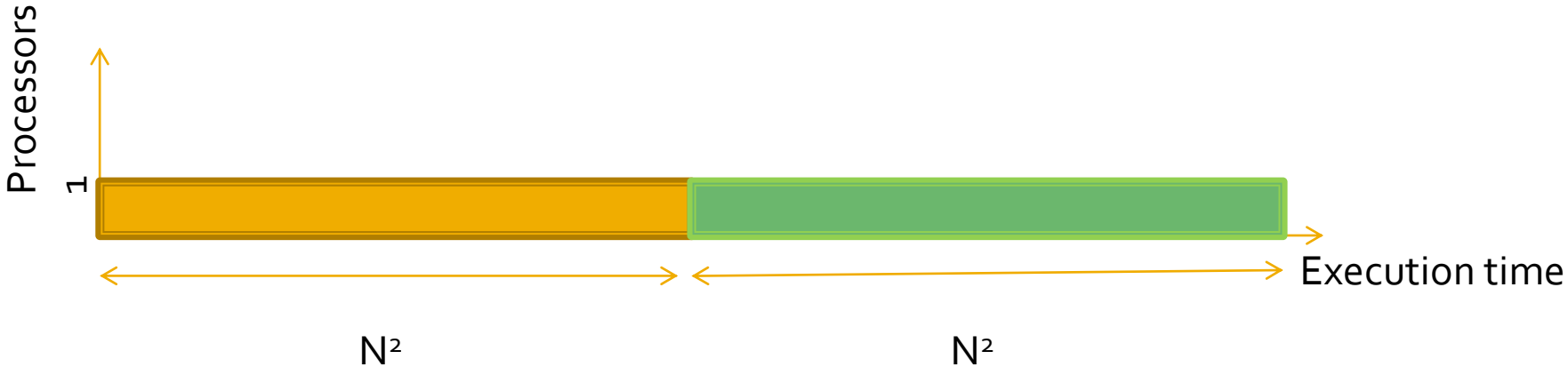
1. Decomposition
2. Assignment
3. Orchestration
4. Mapping

# Decomposition

- Break up a computation into a collection of tasks
- When?
  - Statically
  - Tasks can be created dynamically
- How many?
  - At least enough tasks to occupy all execution units of the physical machine
  - Not too many => create large overhead
- Are there any dependencies?

# Simple example

- 2 computations on a NxN matrix of numbers
  - Add 1 to all elements
  - Sum all the elements
- On one processor

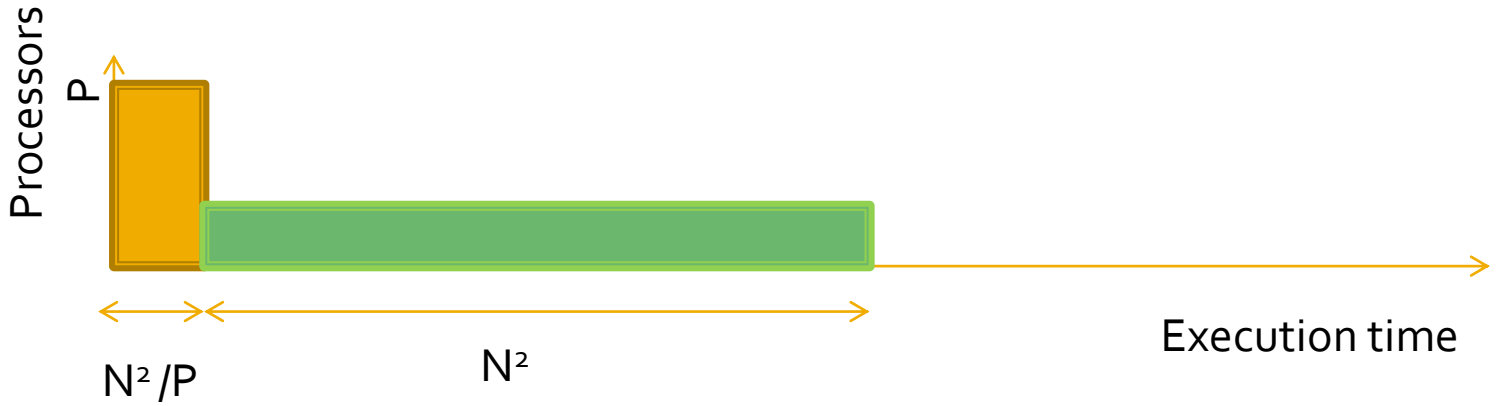


# Simple example

- P processors
- Solution 1:
  - Assign $N^2/P$ numbers to P processors
  - Step 1: all P processors perform $1^{st}$ computation
  - Step 2: each processor adds each of its assigned numbers to a global sum
- Which is the problem?
- Compute the speedup!

# Simple example

- Solution 1




- Parallel execution time: $N^2/P+N^2$

# Simple example

- Solution 2:
  - Step2: Each processor computes the sum of assigned numbers locally (in parallel)
  - Step 3: Add the partial sums (sequential)


- Parallel execution time: $N^2/P+N^2/P+P$
- Compute the speedup!
- Recall Amdahl's law?

# Assignment

- Tasks ("work to do") are distributed to threads ("workers")
- Goals:
  - Balance the workload among processes (load balancing)
  - Reduce communication between threads
  - Reduced overheads introduced by managing the assignment
- When?
  - Static
  - Dynamic

# Partitioning

- Partitioning = Decomposition + Assignment
- Usually independent of the underlying architecture
- Who?
  - Decomposition: often done by programmer
  - Assignment: often in the responsibility of languages/runtimes

# Orchestration

- Involves:
  - Structuring communication
  - Adding synchronization to preserve dependencies if necessary
  - Organizing data structures in memory
  - Scheduling tasks (order of task execution)
- Dependent of the architecture and programming model
- Goals: reduce comm/sync costs, preserve locality of data, reduce overhead

# Mapping

- Map threads to actual HW execution units
- Who?
  - Operating system
  - Compiler
  - Hardware
  - Programmer?
- Example: to reduce communication cost , place cooperating threads on the same processor

# Decompose computation or data?

- Many problems need parallelization because they process a lot of data
- Parallelize programs = partitioning computation?
- Partitioning data: it is equally valid!

# Goals of the parallelization process

| Step | Architecture-dependent? | Major performance goals |
|---|---|---|
| Decomposition | Mostly no | Expose enough concurrency but not too much |
| Assignment | Mostly no | Balance workload<br>Reduce communication volume |
| Orchestration | Yes | Reduce non-inherent communication via data locality<br>Reduce communication and synchronization cost as seen by the processor<br>Reduce serialization at shared resources<br>Schedule tasks to satisfy dependencies early |
| Mapping | Yes | Put related processes on the same processor if necessary<br>Exploit locality in network topology |

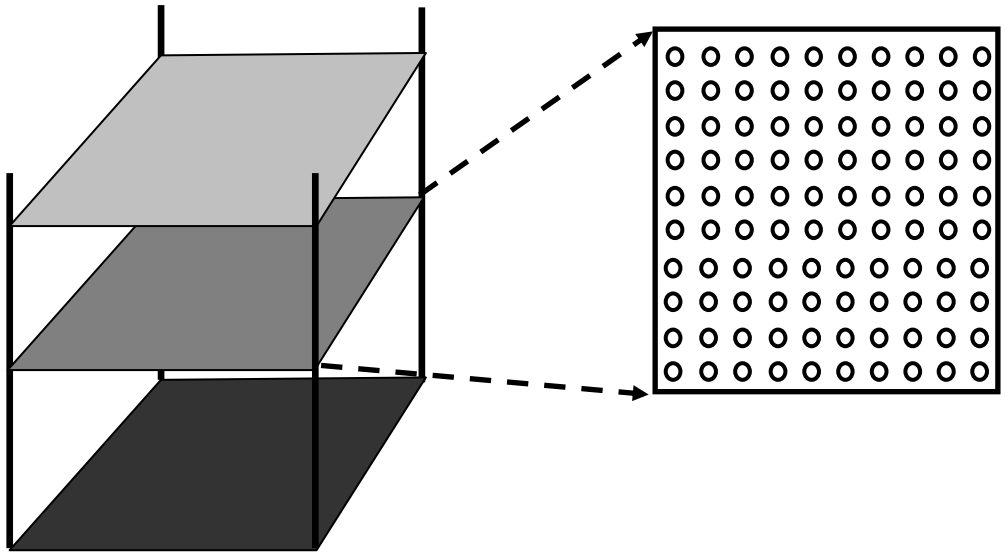
# Example – Ocean currents simulation (from Culler et. al. - Ch 2)

- Simulate the motion of the water currents in the ocean
- Models of ocean behavior and water currents over time

- Problem is continuous in space and time → make problem discrete in space and time
  - How to make space be discrete?
  - How to make time be discrete?

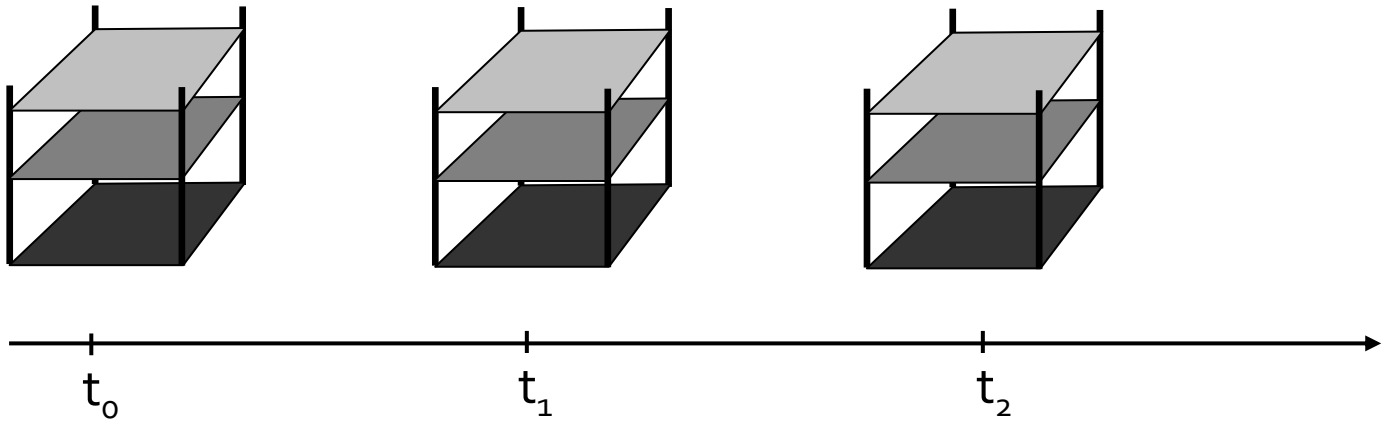
# Ocean simulation

- How to make space be discrete?
  - model the ocean as a grid of points -> set of two-dimensional grids
  - every important variable (pressure, velocity, etc) has a value at each point



# Ocean simulation

- How to make time be discrete?
  - time is made discrete into a series of finite time-steps
  - equations of motion are solved for each point in one time-step, the value of variables are updated, and the equations are solved again
  - every time-step consists of several operations (e.g. set up values for variables using the results from previous step, solve the equations, etc)



# Ocean simulation

- The more grid points, the more accurate the simulation
  - Atlantic: 2 000 km x 2 000 km, using grids of 100 x 100 points implies 20 km between points
- The shorter interval between steps, the more accurate simulation
  - to simulate 5 years of ocean movement, updating every 8 hours implies 5 500 steps

# Ocean simulation

- Simplified version of a part of Ocean application i.e. equation solver (called kernel)
- Illustrate the parallelization using:
  - shared memory
  - message passing
- Decomposition and assignment are the same for the two models

# Kernel solver

- Grid: (n+2) x (n+2) - border rows and columns contain boundary values that do not change, the rest of n x n points are updated by the solver
- Computation for each point:
  - replace its value with a weighted average of itself and its four nearest-neighbor points
  - compute for each point the difference of updated value from its old value
  - if the average difference over all elements is smaller than a predefined value → the solution has converged → kernel stops, otherwise another sweep
- Update computation for a point sees the new values of the points above and to the left and the old values of the points below and to the right

```
1.  int n;                                  /*size of matrix: (n + 2-by-n + 2) elements*/
2.  float **A, diff = 0;

3.  main()
4.  begin
5.    read(n) ;                             /*read input parameter: matrix size*/
6.    A ← malloc (a 2-d array of size n + 2 by n + 2 doubles);
7.    initialize(A);                        /*initialize the matrix A somehow*/
8.    Solve (A);                            /*call the routine to solve equation*/
9.  end main

10. procedure Solve (A)                     /*solve the equation system*/
11.   float **A;                            /*A is an (n + 2)-by-(n + 2) array*/
12. begin
13.   int i, j, done = 0;
14.   float temp;
15.   while (!done) do                      /*outermost loop over sweeps*/
16.     diff = 0;                           /*initialize maximum difference to 0*/
17.     for i ← 1 to n do                   /*sweep over nonborder points of grid*/
18.       for j ← 1 to n do
19.         temp = A[i,j];                  /*save old value of element*/
20.         A[i,j] ← 0.2 * (A[i,j] + A[i,j-1] + A[i-1,j] +
21.           A[i,j+1] + A[i+1,j]);         /*compute average*/
22.         diff += abs(A[i,j] - temp);
23.       end for
24.     end for
25.     if (diff/(n*n) < TOL) then done = 1;
26.   end while
27. end procedure
```

# Kernel solver – decomposition

- Straightforward: program structure-based approach
  - start from the individual loop or loop nests in the program
  - check if their iterations can be performed in parallel
  - check if enough concurrency is exposed
  - look for concurrency across loops

# Kernel solver – decomposition

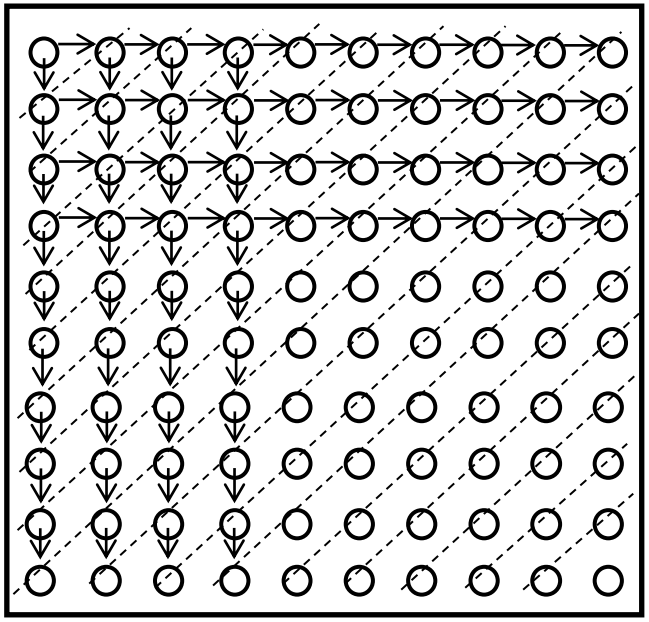
- Decompose work in tasks; each task updates a single grid point
- Ways to exploit exposed concurrency:
  - A. keep loop structure and insert point-to-point synchronization
  - B. change the loop structure
  - C. exploit knowledge of problem beyond the dependencies in the sequential program
  - D. ignore the dependencies among grid points within a sweep, global synchronization between sweeps

# Kernel solver – decomposition

- A. Keep loop structure and insert point-to-point synchronization
  - by point-to-point synchronization it is ensured that the new value for a grid point has been produced in the current sweep before it is used by the points below or to its right
  - different loops and sweeps might be in progress at the same time on different elements
  - disadvantage: synchronization overhead

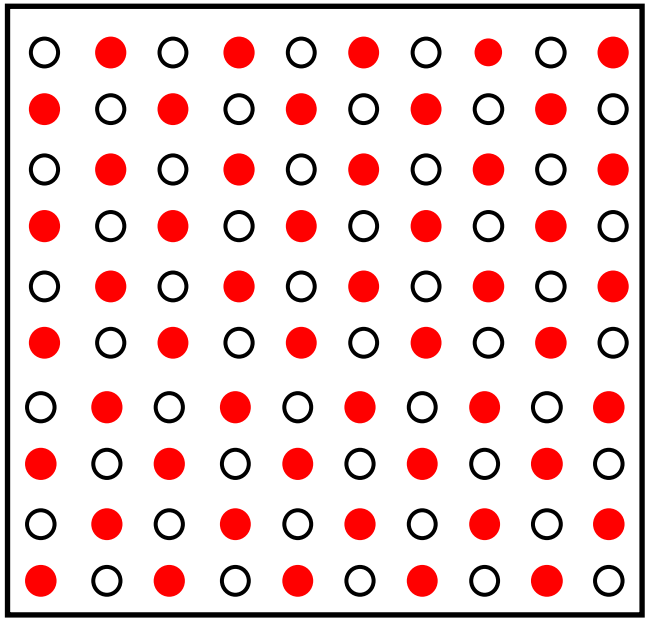
# Kernel solver – decomposition

- B. Change the loop structure
  - check fundamental dependencies in the algorithm



# Kernel solver – decomposition

- C. Exploit knowledge of problem beyond the dependencies in the sequential program



# Kernel solver – decomposition

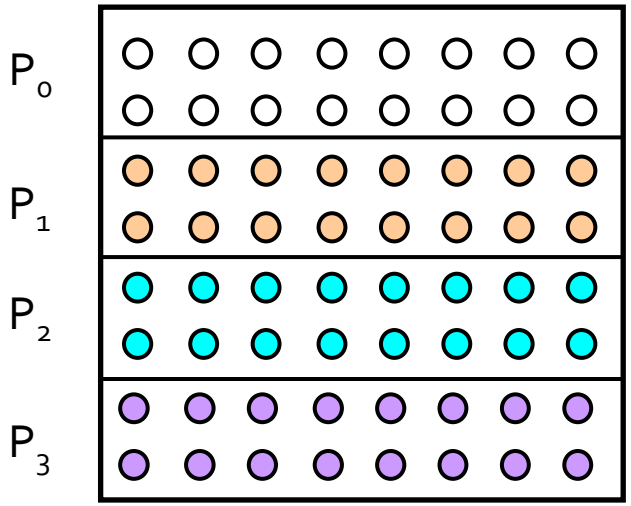
- D. Ignore the dependencies among grid points within a sweep, global synchronization between sweeps

```
15. while (!done) do                          /*a sequential loop*/
16.   diff = 0;
17.   for_all i ← 1 to n do                  /*a parallel loop nest*/
18.     for_all j ← 1 to n do
19.       temp = A[i,j];
20.       A[i,j] ← 0.2 * (A[i,j] + A[i,j-1] + A[i-1,j] +
21.         A[i,j+1] + A[i+1,j]);
22.       diff += abs(A[i,j] - temp);
23.     end for_all
24.   end for_all
25.   if (diff/(n*n) < TOL) then done = 1;
26. end while
```

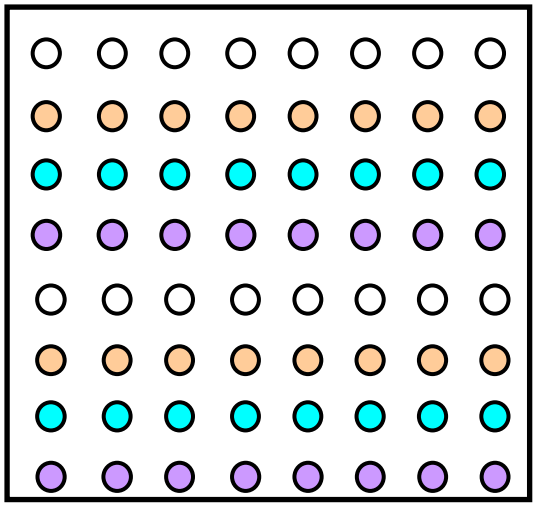
# Kernel solver – assignment

- P processes (threads)
- Each process takes N/P

Block




Interleaved



# Kernel solver – orchestration (DP)

```
1. int n, nprocs;                    /* grid size (n+2-by-n+2) and number of processes*/
2. float **A, diff = 0;

3. main()
4. begin
5.    read(n); read(nprocs);;        /* read input grid size and number of processes*/
6.    A ← G_MALLOC (a 2-d array of size n+2 by n+2 doubles);
7.    initialize(A);                 /* initialize the matrix A somehow */
8.    Solve (A);                     /* call the routine to solve equation*/
9. end main

10. procedure Solve(A)               /* solve the equation system */
11.   float **A;                     /* A is an n+2 by n+2 array*/
12. begin
13.   int i, j, done = 0;
14.   float mydiff = 0, temp;
14a.  DECOMP A[BLOCK,*];
15.   while (!done) do               /* outermost loop over sweeps */
16.         mydiff = 0;              /* initialize maximum difference to 0 */
17.         for_all i ← 1 to n do    /* sweep over non-border points of grid */
18.               for_all j ← 1 to n do
19.                     temp = A[i,j];        /* save old value of element */
20.                     A[i,j] ← 0.2 * (A[i,j] + A[i,j-1] + A[i-1,j] +
21.                           A[i,j+1] + A[i+1,j]);       /*compute average */
22.                     mydiff += abs(A[i,j] - temp);
23.               end for_all
24.         end for_all
24a.        REDUCE (mydiff, diff, ADD);
25.         if (diff/(n*n) < TOL) then done = 1;
26.   end while
27.end procedure
```

# Kernel solver – orchestration (SM)

- matrix A is declared as a shared array
- processes refer to A by load and store primitives

| Name | Syntax | Function |
|---|---|---|
| CREATE | CREATE(p, proc, args) | Create p processes that start executing at procedure **proc** with arguments. |
| G_MALLOC | G_MALLOC(size) | Allocate shared data of **size** bytes. |
| LOCK | LOCK(name) | Acquire mutually exclusive access. |
| UNLOCK | UNLOCK(name) | Release mutually exclusive access. |
| BARRIER | BARRIER(name, number) | Global synchronization among **number** processes: none gets past BARRIER until **number** have arrived. |
| WAIT_FOR_END | WAIT_FOR_END(number) | Wait for **number** processes to terminate. |
| wait for flag | while(!flag); or WAIT(flag) | Wait for **flag** to be set (spin or block); used for point-to-point event synchronization. |
| set flag | flag = 1; or SIGNAL(flag) | Set **flag**; wakes up process that is spinning or blocked on **flag**, if any. |

```
1.  int n, nprocs;              /* matrix dimension and number of processors to be used */
2a. float **A, diff;            /*A is global (shared) array representing the grid */
                                /* diff is global (shared) maximum difference in current sweep */
2b. LOCKDEC(diff_lock);         /* declaration of lock to enforce mutual exclusion */
2c. BARDEC (bar1);              /* barrier declaration for global synchronization between sweeps*/

3.  main()
4.  begin
5.     read(n);    read(nprocs);     /* read input matrix size and number of processes*/
6.     A ← G_MALLOC (a two-dimensional array of size n+2 by n+2 doubles);
7.     initialize(A);                          /* initialize A in an unspecified way*/
8a.    CREATE (nprocs-1, Solve, A);
8      Solve(A);                       /* main process becomes a worker too*/
8b.    WAIT_FOR_END;                   /* wait for all child processes created to terminate */
9.  end main

10. procedure Solve(A)
11. float **A;          /* A is entire n+2-by-n+2 shared array, as in the sequential program */
12. begin
13.    int i,j, pid, done = 0;
14.    float temp, mydiff = 0;              /* private variables /
14a.   int mymin ← 1 + (pid * n/nprocs);        /* assume that n is exactly divisible by  */
14b.   int mymax ← mymin + n/nprocs - 1;        /* nprocs for simplicity here*/

15. while (!done) do                        /* outer loop over all diagonal elements */
16.    mydiff = diff = 0;                   /* set global diff to 0 (okay for all to do it) */
17.    for i ← mymin to mymax do            /* for each of my rows */
18.          for j ← 1 to n do              /* for all elements in that row */
19.                temp = A[i,j];
20.                A[i,j] = 0.2 * (A[i,j] + A[i,j-1] + A[i-1,j] +
21.                      A[i,j+1] + A[i+1,j]);
22.                mydiff += abs(A[i,j] - temp);
23.          endfor
24.    endfor
25a.   LOCK(diff_lock);                 /* update global diff if necessary */
25b.         diff += mydiff;
25c.   UNLOCK(diff_lock);
25d.   BARRIER(bar1, nprocs);                /* ensure all have got here before checking if done*/

25e.   if (diff/(n*n) < TOL) then done = 1; /* check convergence; all get same answer*/
25f.   BARRIER(bar1, nprocs);          /* see Exercise c */
26. endwhile
27.end procedure
```

# Kernel solver – orchestration (MP)

- matrix A is represented by a collection of smaller data structures distributed among the processes

| Name | Syntax | Function |
|---|---|---|
| CREATE | CREATE(procedure) | Create p process that starts at **procedure**. |
| SEND | SEND(src_addr, size, dest, tag) | Send **size** bytes starting at **src_addr** to the **dest** process, with **tag** identifier. |
| RECEIVE | RECEIVE(buffer_addr, size, src, tag) | Receive a message with the **tag** identifier from the **src** process, and put **size** bytes of it into buffer starting at **buffer_addr**. |
| SEND_PROBE | SEND_PROBE (tag, dest) | Check if message with identifier **tag** has been sent to process **dest** (only for asynchronous message passing). |
| RECV_PROBE | RECV_PROBE(tag, src) | Check if message with identifier **tag** has been received from process **src** (only for asynchronous message passing). |
| BARRIER | BARRIER(name, number) | Global synchronization among **number** processes: none gets past **BARRIER** until number have arrived. |
| WAIT_FOR_END | WAIT_FOR_END(number) | Wait for **number** processes to terminate. |

```
1. int pid, n, nprocs;          /* process id, matrix dimension and number of processors to be used */
2. float **myA;
3. main()
4. begin
5.    read(n);  read(nprocs);          /* read input matrix size and number of processes*/
8a.   CREATE (nprocs-1 processes that start at procedure Solve);
8b.   Solve();                         /* main process becomes a worker too*/
8c.   WAIT_FOR_END;                    /* wait for all child processes created to terminate */
9. end main

10. procedure Solve()
11. begin
13.   int i,j, pid, n' = n/nprocs, done = 0;
14.   float temp, tempdiff, mydiff = 0;       /*private variables/
6.    myA ← malloc(a 2-d array of size [n/nprocs + 2] by n+2);/* my assigned rows of A */
7.    initialize(myA);                  /* initialize my rows of A, in an unspecified way*/

15.while (!done) do
16.   mydiff = 0;                              /* set local diff to 0 */
16a.if (pid != 0) then SEND(&myA[1,0],n*sizeof(float),pid-1,ROW);
16b.  if (pid = nprocs-1) then SEND(&myA[n',0],n*sizeof(float),pid+1,ROW);
16c.  if (pid != 0) then RECEIVE(&myA[0,0],n*sizeof(float),pid-1,ROW);
16d.  if (pid != nprocs-1) then RECEIVE(&myA[n'+1,0],n*sizeof(float),pid+1,ROW);
          /* border rows of neighbors have now been copied into myA[0,*] and myA[n'+1,*] */
17.   for i ← 1 to n' do                       /*for each of my rows */
18.         for j ← 1 to n do                  /*for all elements in that row */
19.               temp = myA[i,j];
20.               myA[i,j] ˜ 0.2 * (myA[i,j] + myA[i,j-1] + myA[i-1,j] +
21.                     myA[i,j+1] + myA[i+1,j]);
22.               mydiff += abs(myA[i,j] - temp);
23.         endfor
24.   endfor
      /* communicate local diff values and obtain determine if done; can be replaced by reduction and broadcast */
25a.  if (pid != 0) then                       /*process 0 holds global total diff*/
25b.        SEND(mydiff,sizeof(float),0,DIFF);
25c.        RECEIVE(mydiff,sizeof(float),0,DONE);
25d.  else
25e.        for i ← 1 to nprocs-1 do           /*for each of my rows */
25f.              RECEIVE(tempdiff,sizeof(float),*,DONE);
25g.              mydiff += tempdiff;          /*accumulate into total */
25h.        endfor
25i.        for i ← 1 to nprocs-1 do           /*for each of my rows */
25j.              SEND(done,sizeof(int),i,DONE);
25k.        endfor
25l.  endif
26.         if (mydiff/(n*n) < TOL) then done = 1;
27. endwhile
28. end procedure
```